EPFL - Computer Science

CS-401 Applied Data Analysis

Lecture 09 — 12/11/2025

Author: Jonathan Dutoit

Professor: Maria Brbic

Unsupervised Learning

Unsupervised: Given only samples X of the data, we compute a function f such that $y = f(X)$ is a "simpler" representation:

- Discrete y (cluster labels): **"clustering"**
- Continuous y" **"dimensionality reduction"** e.g., matrix factorization, unsupervised neural networks)

The clustering problem

- Given a set of points, with a notion of distance between points, group the points into some number of ***clusters***, such that
  - members of a cluster are close (i.e., similar) to each other
  - members of different clusters are far apart from each other
- **Usually**:
  - Points live in a high-dimensional space
  - Similarity is defined via a distance measure (Euclidean cosine, Jaccard, edit distance ...)

Characteristics of clustering methods

Clustering algorithms have the following properties:

Quantitative: scalability (many samples), dimensionality (many features)

Qualitative: types of features (numerical, categorical, etc.), type of shapes (polyhedra, hyperplanes, manifolds, etc.)

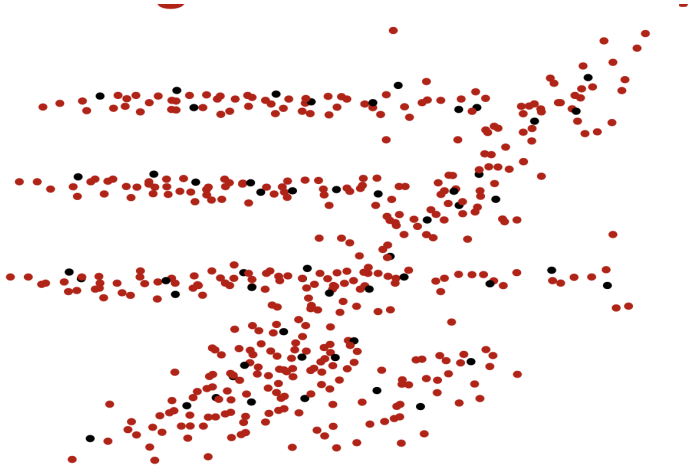
Robustness: sensitivity to noise and outliers, sensitivity to the processing order

User interaction: incorporation of user constraints (e.g., number of clusters, max size of clusters), interpretability and usability.

Some use cases for clustering

- Data exploration (especially for high-dimensional data, where visualization fails)
- Partitioning of data for more fine-grained subsequent analysis
- Marking: building personas
- Supporting data labeling for supervised learning
- Supporting feature discretization for supervised learning
- Data compression
- ...

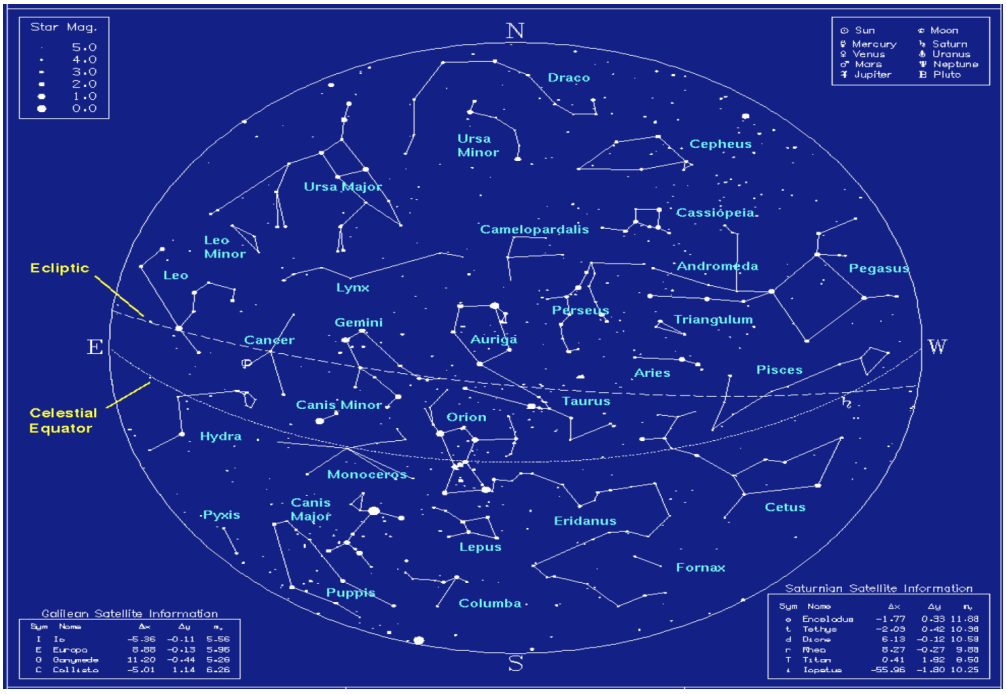
Clustering for condensation/compression



In the picture above, the black dots are "class representatives" of the data. We don't require that clusters extract meaningful structure, but that they give a coarse-grained version of the data.

Beware of "cluster bias"!

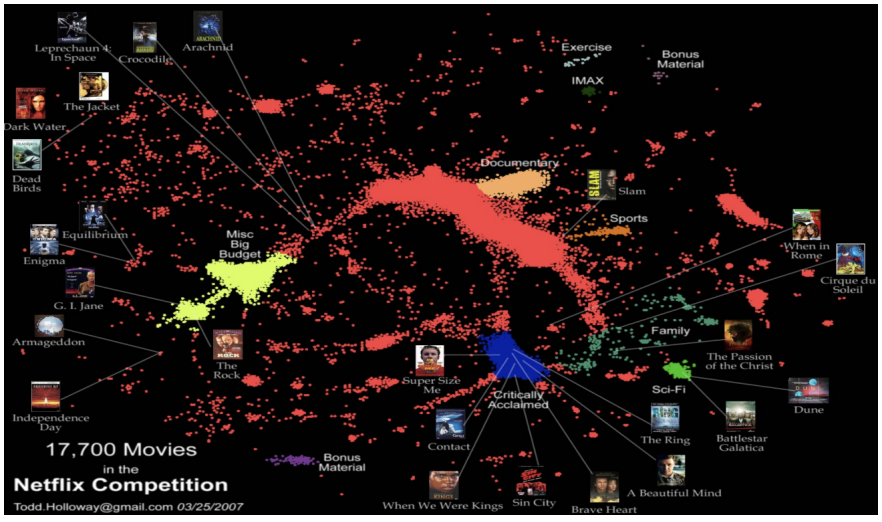
- Human beings conceptualize the world through categories represented as exemplars or prototypes
- We tend to see cluster structure whether it is there or not.
- Works well for dogs, but...



Even if we see structures/figures, it doesn't mean that statistically it makes sense.

Cluster bias

- **Clustering is used more than it should be**, because people assume an underlying domain has discrete classes in it
  - Especially true for characteristics of people, e.g., Myers-Briggs personality types like "ENTP".
- In reality the underlying data is often **continuous**
- In such cases, continuous models (e.g., matrix factorization, "soft" clustering where we use probabilities) tend to do beter



In this Netflix dataset, each data point is a movie that is represented by a vector of real numbers based on user's watching it or not. There are million of users, hence this was dimensionality reduce to be able to see it in 2D.

Note that the closer movies are in this space, the more similar they are too.

- Central portion: more of a continuum than discrete cluster
- Other methods (e.g., dimensionality reduction) may be more appropriate than discrete clustering methods.

Terminology

- Hierarchical clustering: cluster form a tree-shaped hierarchy. Can be computed bottom-up or top-down
- Flat clustering: no inter-cluster structure
- Hard clustering: items assigned to a unique cluster
- Soft clustering: cluster membership is a probability distribution over all clusters

Clustering is hard

- Clustering small amounts of data (like 2 dimensions) looks easy, and actually is often easy but...
- ... many applications involve not 2, but hundreds or thousands of dimensions (and large amounts of data)
- High-dimensional spaces are different ("***curse of dimensionality***"): volume grows exponentially with the number of dimensions; space is sparsely populated; clusters become less tight.

Clustering problem: galaxies

- A catalog of 2 billion "sky objects" represents objects by their radiation in 7 dimensions (frequency bands)
- Problem: cluster into similar objects, e.g., galaxies, nearby stars, quasars, etc.
- Sloan Digital Sky Survey

Clustering problem: movies

- **Intuitively: Movies divide into categories/genres, and customers prefer a few categories**
  - But what are categories really?
  - $\Rightarrow$ take a data-driven approach!
- Represents a movie by a set of customers who watched it ("collaborative filtering")
- **Idea**: Similar movies have similar sets of customers, and vice-versa

Space of all movies:

- Think of a space with one dimension for each customer
  - Values in a dimension may be 0 or 1 only
  - A movie is a point in this space $(x_1, x_2, ..., x_n)$ where $x_i = 1$ iff the i=th customer watched the movie
- For Amazon/Netflix, the dimensionality is in the millions
- **Task**: Find clusters of similar movies

Cosine, Jaccard, Euclidean distances

In all examples, we have a choice when thinking of data points as sets of features:

- Sets as vectors
  - Measure similarity via **Euclidian distance** or **cosine distance**
- Sets as sets:
  - Measure similarity via **Jaccard index**

Overview: Methods of clustering

Hierarchical methods:"

- **Agglomerative** (bottom-up):
  - Initially, each point is a cluster
  - Repeatedly combine two "nearest" clusters into one
- **Divisive** (top-down):
  - Start with one cluster and recursively split it

Flat methods (a.k.a point-assignment methods):

- Maintain a set of clusters
- Assign points to "nearest" cluster; recompute clusters; repeat

Agglomerative hierarchical clustering

Key operation: repeatedly combine two nearest clusters

Three important questions:

- (1) How to represent a cluster of more than one point?
  - Euclidean case: represent a cluster via its **centroid** = average of points in cluster
  - What about non-Euclidean case?
- (2) How to determine the "nearness" of clusters?
  - Euclidean case: distance between clusters = distance between centroids
  - What about non-Euclidean case?

- (3) When to stop combining clusters?

Non-Euclidean case: clustroids

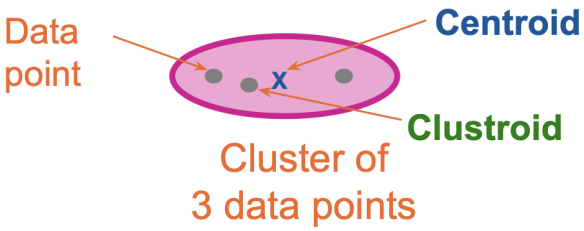
(1) Howe to represent a cluster of many points?

clustroid = actual data point that is **"closest"** to the other points

- Possible meanings of "closest"
  - Smallest average distance to other points (a.k.a **medoid**)
  - Smallest sum of squares of distances to other points
  - Smallest maximum distance to other points

Centroid is the average of all data points in the cluster. This means centroid is an "artificial" point

clustroid is an **existing** data point that is "closest" to all other points in the cluster.



(2) How do you determine the "nearness" of clusters?

- Approach 1:
  Intercluster distance = minimum of the distances between any two points, one from each cluster; or average of distances; or distance between clustroids; etc.
- Approach 2:
  Pick a notion of "**cohesion**" ("tightness") of a cluster
  Then: nearness of clusters = cohesion of their *union*

Cohesion

- **Approach 2.1**: Use the **diameter** of the merged cluster = maximum distance between points in the merged cluster
- **Approach 2.2**: Use the **average distance** between points in the merged cluster

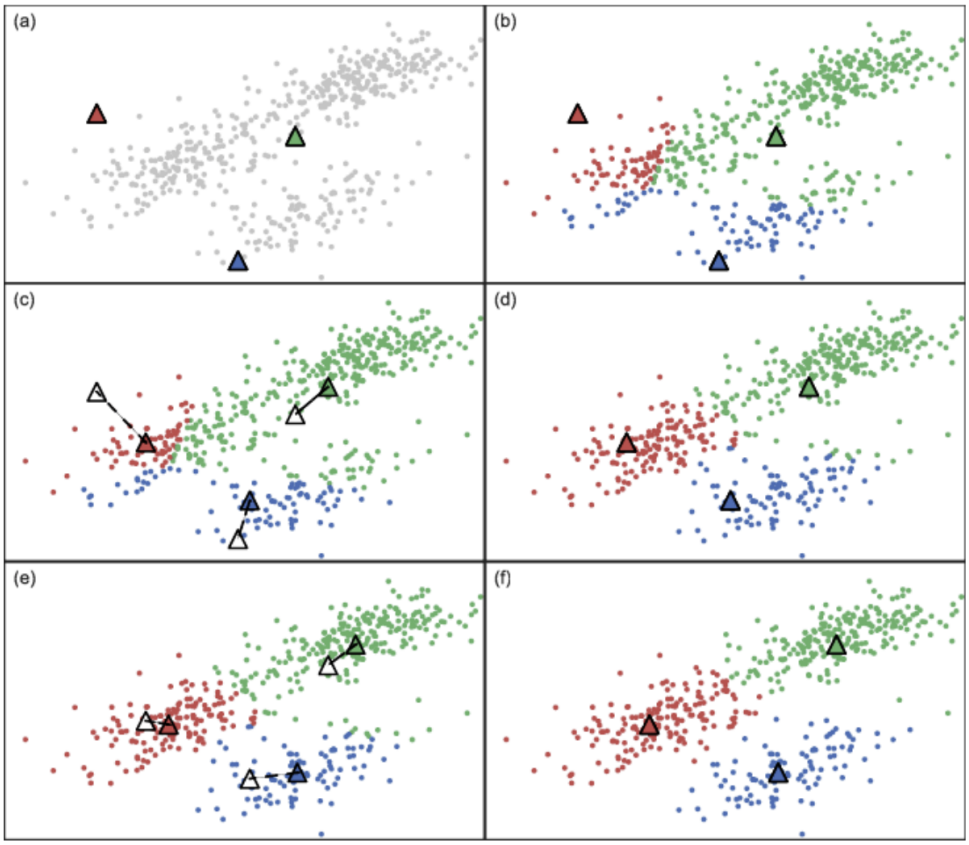
Implementation

- **Naïve implementation of hierarchical clustering**:
  - At each step, compute pairwise distance between all pairs of clusters, then merge
  - $O(N^3)$, where N is the number of data points
- **Careful implementation using priority queue** can reduce time to $O(N^2 \log N)$
  - Still too expensive for really big datasets

K-means

Goal: assign each data point to one of k clusters such that the total distance of points to their centroids is minimized

- Solved by a simple greedy algorithm (Lloyd's algorithm):
  Locally minimize the "distance" (usually squared Euclidean distance) from data points to their respective centroids:
  - **Find the closest cluster centroid** for each data point, and assign the point to that cluster
  - **Recompute the cluster centroid** (the mean of data points in the cluster) for each cluster.



How long to iterate?

- For fixed number of iterations
- or until no change in assignments (guaranteed to happen)
- or until only small change in cluster "tightness" (sum of [squared] distances from points to centroids)

K-means initialization

We need to pick some points for the first round of the algorithm:

- **Random sample**: Pick a random subset of k points from the dataset.
- **K-Means++**: Iteratively construct a random sample with good spacing across the dataset.

Note: Finding an optimal k-means clustering is NP-hard. The above help avoid bad configurations. K-means is a **greedy algorithm**.

K-means++

Start: Choose first cluster center at random from the data points

Iterate:

- For every remaining data point x, compute the distance $D(x)$ from x to the closest previously selected cluster center.
- Choose a remaining point x randomly with probability proportional to $D(x)^2$, and make it a new cluster center.

Intuitively, this finds a sample of widely-spaced points from dense regions of the data space, avoiding "collapsing" of the clustering into a few internal centers.

K-means properties

- Greedy algorithm with random initialization - **solution may be suboptimal** and vary significantly with different initial points
- Very simple convergence proofs
- **Performance is *O(nk)* per iteration** - not bad, and ca be heuristically improved.
  n = number of points in the dataset, k = number clusters
- Many variants, e.g., Fixed-size clusters, soft clustering
- Works well for data condensation/compression

K-means drawbacks

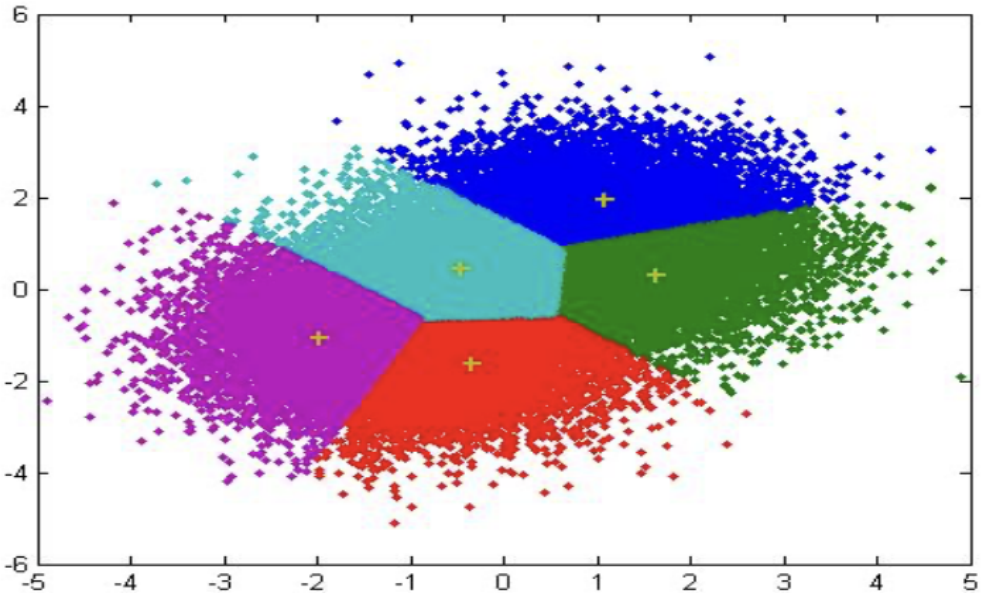
Often terminates at **local but non-global optimum** (mitigated by smart initialization such k-means++, or by re-running multiple times with different initializations)

Requires the notion of a **mean**

Requires specifying ***k*** (number of clusters) in advance

Doesn't handle **noisy data and outliers** well

Clusters **only** have **convex shapes**



How to choose number of clusters k?

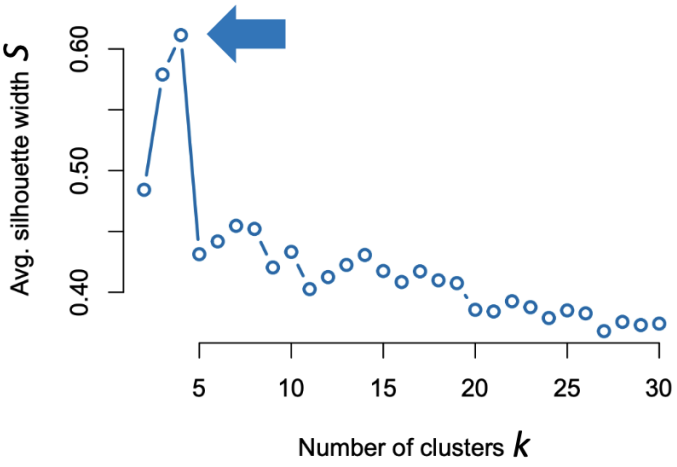
For k = 1, 2, 3, ...

- Run k-means with k clusters
- For each data point i, compute "silhouette width"

$$s(i) = \frac{b(i) - a(i)}{max\{a(i), b(i)\}}.$$

  where *b(i)*: avg. distance to points in closest other cluster, and *a(i)*: avg. distance to points in own cluster

- S = average of *s(i)* over all i
- Plot S against k
- Pick k which S is greatest

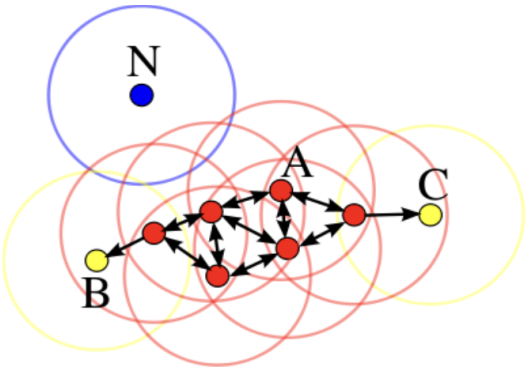


DBSCAN

"Density-based spatial clustering of applications with noise"

Motivation: centroid-based clustering methods like k-means favor clusters that are spherical, and have great difficulty with anything else

DBSCAN performs density-based clustering, and follows the shape of dense neighborhoods of pints.



- **Def.: core points** have a lest *minPts* neighbors in a sphere of diameter ϵ around them.
- The **red** points here are core points with at least $minPts = 3$ neighbors in an ϵ-sphere around them.
- Core points can **directly reach** neighbors in their ϵ-sphere
- From non-core points, no other points can be reached
- Points q is **density-reachable** from p if there is a series of points $p = p_1, ..., p_n = q$ such that p_{i+1} is directly reachable from p_i (yellow points)
- All points not density-reachable from any other points are **outliers/noise** (blue points)
- Points p, q are **density-connected** if there is a point o such that both p and q are density-reachable from o.
- A **cluster** is a set of points that are **mutually density-connected**.
- That is, if a point is density-reachable from a cluster point, it is part of the cluster as well
- In the above figure, red points are mutually density-reachable; B and C are density-connected; N is an outlier.

DBSCAN performance

- DBSCAN uses all-pairs point distances, but using an efficient indexing structure, each Range-Query (for finding neighbors within ϵ-sphere) takes only $O(\log n)$ time
- The algorithm overall can be made to run in $O(n \log n)$
- Fast neighbor search becomes progressively harder (higher constants) in higher dimensions, due to the **curse of dimensionality**!